

NiceGUI 中 ui.aggrid 组件全面解析

NiceGUI 的 `ui.aggrid` 组件基于强大的 AG Grid 构建，是一款专为复杂数据场景设计的高级网格组件，支持海量数据处理、灵活的列配置、丰富的交互功能及深度自定义能力。相较于基础的 `ui.table`，`ui.aggrid` 更适合企业级应用、大数据展示、复杂数据编辑等场景，提供了 AG Grid 原生的核心能力与 NiceGUI 简洁 API 的完美结合。以下从核心概念、基础用法、高级特性、数据集成及 API 详解等方面进行全面阐述。

一、核心概念与基础参数

1. 组件本质

`ui.aggrid` 通过 `options` 参数配置 AG Grid 的核心属性（列定义、行数据、交互规则等），支持通过 API 方法与网格实例实时交互，所有配置变更可通过修改属性或调用方法即时生效，无需手动重建组件。其核心优势在于 AG Grid 原生的高性能渲染、丰富的筛选排序能力及灵活的扩展机制。

2. 基础参数说明

参数名	类型	说明	默认值	关键备注
<code>options</code>	字典	AG Grid 核心配置，包含 <code>columnDefs</code> （列定义）、 <code>rowData</code> （行数据）等	-	必选，完全兼容 AG Grid 原生配置
<code>html_columns</code>	列表（整数）	需要渲染为 HTML 的列索引列表（按列定义顺序）	<code>[]</code>	支持单元格内 HTML 内容（如链接、按钮）
<code>theme</code>	字符串	网格主题，可选 值："quartz"、"balham"、"material"、"alpine"	优先取 <code>options['theme']</code> ，否则为 "quartz"	支持动态切换，自动适配页面深色模式
<code>auto_size_columns</code>	布尔值	是否自动调整列宽以适应网格宽度	<code>True</code>	关闭后需手动配置列宽或启用列拖动调整

3. 核心配置 (options) 详解

`options` 参数是 `ui.aggrid` 的核心，完全兼容 AG Grid 原生配置，关键子属性如下：

- `columnDefs`：列定义列表，每个元素为字典，支持 `headerName`（列标题）、`field`（行数据键名）、`filter`（筛选器类型）、`sortable`（是否支持排序）、`editable`（是否可编辑）等；
- `rowData`：行数据列表，每个元素为字典，键名与 `columnDefs` 的 `field` 对应，支持嵌套对象（通过 `.` 分隔字段名，如 `name.first`）；
- `rowSelection`：行选择配置，字典类型，`mode` 可选 "single"（单选）、"multiRow"（多选）；
- `cellClassRules`：单元格条件样式规则，字典类型，键为 CSS 类名，值为 JavaScript 表达式（基于单元格值 `x` 判断）；
- `:getRowHeight`：动态行高函数（前缀 `:` 表示 JavaScript 表达式），基于行数据返回行高；
- `:getRowId`：行 ID 生成函数，用于唯一标识行（支持通过行数据字段定义，如 `params.data.name`）。

二、基础用法

1. 最小化示例（基础网格）

通过 `columnDefs` 定义列结构，`rowData` 传入行数据，快速创建基础网格：

```
from nicegui import ui

# 核心配置：列定义+行数据
grid_options = {
    'columnDefs': [
        {'headerName': 'Name', 'field': 'name', 'sortable': True},
        {'headerName': 'Age', 'field': 'age', 'sortable': True},
        {'headerName': 'Parent', 'field': 'parent', 'hide': True}, # 初始隐藏列
    ],
    'rowData': [
        {'name': 'Alice', 'age': 18, 'parent': 'David'},
        {'name': 'Bob', 'age': 21, 'parent': 'Eve'},
        {'name': 'Carol', 'age': 42, 'parent': 'Frank'},
    ],
    'rowSelection': {'mode': 'multiRow'}, # 多选模式
}

grid = ui.aggrid(grid_options)

# 交互按钮：更新数据、全选、显示隐藏列
ui.button('Update Age', on_click=lambda: grid.options['rowData'][0]['age'] += 1)
ui.button('Select All', on_click=lambda: grid.run_grid_method('selectAll'))
ui.button('Show Parent', on_click=lambda: grid.run_grid_method('setColumnsVisible',
    ['parent'], True))

ui.run()
```

2. 从 DataFrame 创建网格

支持通过 `from_pandas` (Pandas) 和 `from_polars` (Polars) 静态方法，直接将 DataFrame 转换为网格，无需手动配置列和行：

(1) Pandas DataFrame

```
import pandas as pd
from nicegui import ui

# 创建Pandas DataFrame
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Carol'],
    'Age': [18, 21, 42],
    'City': ['New York', 'London', 'Paris']
})

# 直接转换为网格（支持添加样式、HTML列等参数）
ui.aggrid.from_pandas(df).classes('max-h-40 w-full')
ui.run()
```

- 注意：DataFrame 中的非序列化类型（如 `datetime64[ns]`、`complex128`）会自动转为字符串，如需自定义转换需提前处理 DataFrame。

(2) Polars DataFrame (2.7.0 + 新增)

```
import polars as pl
from nicegui import ui

# 创建Polars DataFrame
df = pl.DataFrame({
    'Name': ['Alice', 'Bob', 'Carol'],
    'Age': [18, 21, 42],
    'City': ['New York', 'London', 'Paris']
})

# 直接转换为网格
ui.aggrid.from_polars(df).classes('max-h-40 w-full')
ui.run()
```

- 注意：非 UTF-8 类型字段会自动转为字符串，自定义转换需提前处理。

3. 动态添加行

通过修改 `grid.options['rowData']` 添加行数据，并使用 `ensureIndexVisible` 方法滚动到新增行：

```
import random
from nicegui import ui

def add_row():
    # 新增随机数行
    grid.options['rowData'].append({'number': random.randint(0, 100)})
    # 滚动到最后一行（AG Grid原生API）
    grid.run_grid_method('ensureIndexVisible', len(grid.options['rowData']) - 1)

# 初始为空的网格，开启虚拟滚动优化性能
```

```
grid = ui.aggrid({
    'columnDefs': [{'field': 'number', 'headerName': 'Random Number'}],
    'rowData': [],
}).classes('h-52')

ui.button('Add Row', on_click=add_row)
ui.run()
```

三、核心交互功能

1. 行选择与选中行获取

支持单选 / 多选模式，通过 `get_selected_rows`（获取所有选中行）和 `get_selected_row`（获取首个选中行）方法获取选中数据：

```
from nicegui import ui

grid = ui.aggrid({
    'columnDefs': [{'headerName': 'Name', 'field': 'name'}, {'headerName': 'Age', 'field': 'age'}],
    'rowData': [{'name': 'Alice', 'age': 18}, {'name': 'Bob', 'age': 21}, {'name': 'Carol', 'age': 42}],
    'rowSelection': {'mode': 'multiRow'}, # 多选模式
})

# 获取所有选中行
async def show_selected_rows():
    rows = await grid.get_selected_rows()
    if rows:
        for row in rows:
            ui.notify(f"{row['name']} (Age: {row['age']})")
    else:
        ui.notify('No rows selected')

# 获取首个选中行（适用于单选模式）
async def show_selected_row():
    row = await grid.get_selected_row()
    ui.notify(f"Selected: {row['name']}" if row else 'No row selected')

ui.button('Show All Selected', on_click=show_selected_rows)
ui.button('Show First Selected', on_click=show_selected_row)
ui.run()
```

2. 列筛选 (Mini Filters)

通过在 `columnDefs` 中配置 `filter` 和 `floatingFilter` 属性，为列添加头部筛选器（文本 / 数字筛选）：

```

from nicegui import ui

ui.aggrid({
    'columnDefs': [
        # 文本筛选器: 匹配包含指定字符的内容 (如"a"匹配"Alice"、"Carol")
        {'headerName': 'Name', 'field': 'name', 'filter': 'agTextColumnFilter',
        'floatingFilter': True},
        # 数字筛选器: 精确匹配数字 (如"18"匹配18, 不匹配21)
        {'headerName': 'Age', 'field': 'age', 'filter': 'agNumberColumnFilter',
        'floatingFilter': True},
    ],
    'rowData': [{ 'name': 'Alice', 'age': 18 }, { 'name': 'Bob', 'age': 21 }, { 'name': 'Carol',
    'age': 42 }],
}).classes('w-full')
ui.run()

```

3. 主题切换

通过绑定 `theme` 属性, 动态切换网格主题:

```

from nicegui import ui

grid = ui.aggrid({
    'columnDefs': [{'headerName': 'Make', 'field': 'make'}, {'headerName': 'Country',
    'field': 'country'}],
    'rowData': [
        {'make': 'Ford', 'country': 'USA'},
        {'make': 'Toyota', 'country': 'Japan'},
        {'make': 'Volkswagen', 'country': 'Germany'},
    ],
})

# 单选按钮切换主题
ui.toggle(['quartz', 'balham', 'material', 'alpine']) \
    .bind_value(grid, 'theme').props('flat size="sm"')
ui.run()

```

四、高级自定义特性

1. 条件单元格格式化

通过 `cellClassRules` 配置条件样式, 根据单元格值动态应用 CSS 类 (如年龄 < 21 显示红色背景, ≥21 显示绿色背景):

```

from nicegui import ui

ui.aggrid({
    'columnDefs': [
        {'headerName': 'Name', 'field': 'name'},
        {
            'headerName': 'Age',

```

```

        'field': 'age',
        'cellClassRules': {
            'bg-red-300': 'x < 21', # 年龄<21: 红色背景
            'bg-green-300': 'x >= 21' # 年龄≥21: 绿色背景
        }
    },
],
'rowData': [{ 'name': 'Alice', 'age': 18 }, { 'name': 'Bob', 'age': 21 }, { 'name': 'Carol',
'age': 42 } ],
}).classes('w-full')
ui.run()

```

- 注：CSS 类支持 Tailwind 类（如 `bg-red-300`）或自定义 CSS 类。

2. 渲染 HTML 内容

通过 `html_columns` 参数指定需要渲染为 HTML 的列索引，支持单元格内嵌入链接、按钮等 HTML 元素：

```

from nicegui import ui

# 列索引1（URL列）渲染为HTML链接
ui.aggrid({
    'columnDefs': [
        { 'headerName': 'Name', 'field': 'name' },
        { 'headerName': 'URL', 'field': 'url' },
    ],
    'rowData': [
        { 'name': 'Google', 'url': '<a href="https://google.com"
target="_blank">Google</a>' },
        { 'name': 'GitHub', 'url': '<a href="https://github.com"
target="_blank">GitHub</a>' },
    ],
}, html_columns=[1]).classes('w-full')
ui.run()

```

3. 动态行高

通过 `:getRowHeight` 配置动态行高函数，根据行数据调整行高（如年龄 > 35 时行高为 50px，否则为 25px）：

```

from nicegui import ui

ui.aggrid({
    'columnDefs': [{ 'field': 'name', 'headerName': 'Name' }, { 'field': 'age', 'headerName':
'Age' } ],
    'rowData': [{ 'name': 'Alice', 'age': 18 }, { 'name': 'Bob', 'age': 21 }, { 'name': 'Carol',
'age': 42 } ],
    ':getRowHeight': 'params => params.data.age > 35 ? 50 : 25', # 动态行高逻辑
}).classes('w-full')
ui.run()

```

4. 嵌套对象数据支持

通过 `.` 分隔字段名，支持行数据中的嵌套对象（如 `name.first` 对应 `{'name': {'first': 'Alice'}}`）：

```
from nicegui import ui

ui.aggrid({
    'columnDefs': [
        {'headerName': 'First Name', 'field': 'name.first'}, # 嵌套字段: name.first
        {'headerName': 'Last Name', 'field': 'name.last'},   # 嵌套字段: name.last
        {'headerName': 'Age', 'field': 'age'},
    ],
    'rowData': [
        {'name': {'first': 'Alice', 'last': 'Adams'}, 'age': 18},
        {'name': {'first': 'Bob', 'last': 'Brown'}, 'age': 21},
        {'name': {'first': 'Carol', 'last': 'Clark'}, 'age': 42},
    ],
}).classes('w-full')
ui.run()
```

5. 行方法调用（修改指定行数据）

通过 `run_row_method` 方法，针对特定行执行 AG Grid 原生方法（如修改单元格值），保留行选择状态：

```
from nicegui import ui

grid = ui.aggrid({
    'columnDefs': [{'field': 'name'}, {'field': 'age'}],
    'rowData': [{'name': 'Alice', 'age': 18}, {'name': 'Bob', 'age': 21}, {'name': 'Carol',
'age': 42}],
    ':getRowId': '(params) => params.data.name', # 以name作为行ID
})

# 修改行ID为"Alice"的行的age字段值为99
ui.button('Update Alice\'s Age',
          on_click=lambda: grid.run_row_method('Alice', 'setDataValue', 'age', 99))
ui.run()
```

6. 监听 AG Grid 事件

支持订阅 AG Grid 原生事件（如单元格点击、行选择变更），通过 `on` 方法绑定回调：

```
from nicegui import ui

# 监听单元格点击事件
ui.aggrid({
    'columnDefs': [{'headerName': 'Name', 'field': 'name'}, {'headerName': 'Age', 'field':
'age'}],
    'rowData': [{'name': 'Alice', 'age': 18}, {'name': 'Bob', 'age': 21}, {'name': 'Carol',
'age': 42}],
}).on('cellClicked', lambda e: ui.notify(f'Clicked Cell: {e.args["value"]}'))
```

```
# 监听行选择变更事件
grid = ui.aggrid({
    'columnDefs': [{'headerName': 'Name', 'field': 'name'}],
    'rowData': [{'name': 'Alice'}, {'name': 'Bob'}],
    'rowSelection': {'mode': 'single'},
})
grid.on('selectionChanged', lambda: ui.notify('Selection changed'))

ui.run()
```

- 注：所有 AG Grid 原生事件均可通过 on 方法订阅，事件参数通过 e.args 获取。

7. 筛选方法返回值

通过 `run_grid_method` 调用 AG Grid 方法时，可通过 JavaScript 函数筛选返回结果（如仅获取行数据的 `data` 属性）：

```
from nicegui import ui

grid = ui.aggrid({
    'columnDefs': [{'field': 'name', 'headerName': 'Name'}],
    'rowData': [{'name': 'Alice'}, {'name': 'Bob'}],
})

# 获取第0行的data属性（筛选返回值）
async def get_first_row():
    row_data = await grid.run_grid_method('g => g.getDisplayedRowAtIndex(0).data')
    ui.notify(f'First Row: {row_data["name"]}')

ui.button('Get First Row', on_click=get_first_row)
ui.run()
```

五、关键 API 详解

1. 核心方法

`from_pandas(df, ...)`

- 作用：从Pandas DataFrame快速创建网格，无需手动配置列和行数据。
- 参数：`df` 为必填的Pandas DataFrame；`html_columns` 指定需要渲染为HTML的列索引列表；`theme` 设置网格主题（可选值为"quartz"、"balham"、"material"、"alpine"）；`auto_size_columns` 控制是否自动调整列宽以适应网格宽度（默认True）；`options` 可传入额外的AG Grid配置项。
- 返回值：ui.aggrid网格实例。

`from_polars(df, ...)`

- 作用：从Polars DataFrame创建网格，适配Polars数据结构。
- 参数：与 `from_pandas` 完全一致，仅数据源类型为Polars DataFrame。
- 返回值：ui.aggrid网格实例。

get_selected_rows()

- 作用：获取网格中所有被选中的行数据。
- 参数：无额外参数。
- 返回值：列表类型，每个元素为字典格式的行数据（键名对应列定义的 field）。

get_selected_row()

- 作用：获取网格中首个被选中的行数据，适用于单选模式或仅需首个选中行的场景。
- 参数：无额外参数。
- 返回值：字典格式的行数据，若无选中行则返回None。

run_grid_method(name, *args)

- 作用：调用AG Grid原生API方法，实现高级网格操作。
- 参数：name 为AG Grid原生方法名（如 selectAll 表示全选、setColumnsVisible 表示显示/隐藏列）；*args 为该方法所需的参数（数量和类型与AG Grid原生方法一致）。
- 返回值：AwaitableResponse可等待对象，通过 await 可获取AG Grid方法的返回结果。

run_row_method(row_id, name, *args)

- 作用：针对指定行调用AG Grid行级原生方法，如修改特定行的单元格值。
- 参数：row_id 为行唯一标识（由 options 中的 :getRowId 配置定义）；name 为行级方法名（如 setDataValue 表示修改单元格值）；*args 为方法所需参数。
- 返回值：AwaitableResponse可等待对象，await 后获取方法执行结果。

get_client_data(method)

- 作用：获取客户端网格数据（包含用户在前端的编辑内容），同步前端最新数据到后端。
- 参数：method 指定数据获取方式，可选值为"all_unsorted"（所有未排序数据）、"filtered_unsorted"（筛选后未排序数据）、"filtered_sorted"（筛选并排序后数据）、"leaf"（叶子节点数据，适用于树形网格）。
- 返回值：列表类型，每个元素为字典格式的最新行数据。

load_client_data()

- 作用：将客户端（前端）的编辑数据同步到服务器（后端）的网格实例中，更新 options['rowData']。
- 参数：无额外参数。
- 返回值：无。

2. 可绑定属性

属性名	类型	说明
auto_size_columns	布尔值	是否自动调整列宽（可通过 bind_value 绑定）
html_columns	列表（整数）	HTML 列索引列表（可动态修改）
theme	字符串	网格主题（可通过 bind_value 绑定切换）

属性名	类型	说明
<code>options</code>	字典	AG Grid 配置（修改后需调用 <code>update()</code> 生效）
<code>visible</code>	布尔值	组件可见性（继承自 <code>Element</code> ）

六、注意事项与性能优化

1. 数据格式限制

- 行数据（`rowData`）的键名不能包含 `.`，否则会与嵌套字段解析冲突；
- 非序列化数据类型（如 Pandas 的 `datetime64[ns]`）需提前转换为字符串或其他可序列化类型；
- HTML 列内容需确保安全（避免注入攻击），优先使用可信数据源。

2. 性能优化建议

- 海量数据（万行以上）时，开启虚拟滚动（AG Grid 原生配置 `rowModelType: 'infinite'`），避免一次性渲染所有行；
- 关闭 `auto_size_columns`，手动配置列宽或启用列拖动调整，减少自动计算开销；
- 复杂筛选 / 排序优先使用 AG Grid 原生功能（如 `filter` 配置），而非 Python 端处理，利用客户端计算资源；
- 避免频繁修改 `options` 整体配置，优先修改 `rowData` 或 `columnDefs` 子属性并调用 `update()`。

3. 版本兼容性

- 2.7.0 版本：新增 `from_polars` 方法，支持 Polars DataFrame；
- 2.16.0 版本：新增 `html_id` 属性；
- 2.18.0 版本：`on` 方法支持同时指定 Python handler 和 js_handler；
- 2.19.0 版本：`from_pandas` 和 `from_polars` 方法新增 `html_columns` 参数。

总结

NiceGUI 的 `ui.aggrid` 组件是 AG Grid 强大功能与 NiceGUI 简洁开发体验的完美融合，提供了企业级数据网格所需的全部核心能力：海量数据处理、灵活筛选排序、深度自定义渲染、丰富交互事件等。相较于基础的 `ui.table`，`ui.aggrid` 更适合复杂数据场景（如大数据展示、可编辑表格、多条件筛选），而其 API 设计保持了 NiceGUI 一贯的简洁性，无需深入学习 AG Grid 原生细节即可快速上手。核心优势在于“高性能 + 高灵活度”——既继承了 AG Grid 的性能优化（如虚拟滚动、高效渲染），又支持通过 NiceGUI 的绑定、事件等机制快速集成到应用中，是企业级 NiceGUI 应用中数据展示与交互的首选组件。